Project Report – RAG Chatbot Assignment

Introduction

This project focuses on building a **Retrieval-Augmented Generation (RAG) Chatbot** using open-source language models and vector databases. The system is designed to process a large text document (~10,500+ words), index it for retrieval, and provide factually grounded responses to user queries via a **Streamlit-based chatbot interface**.

Objectives

- Preprocess and ingest a large document for retrieval.
- Chunk the text into semantically coherent segments.
- Generate vector embeddings using a pre-trained embedding model.
- Store embeddings in a vector database (FAISS).
- Implement a retriever-generator pipeline with semantic search.
- Design an effective prompt template to inject context into the LLM.
- Deploy a chatbot with **real-time streaming responses** and **source citation display**.

Document Preparation & Ingestion

Cleaning & Formatting

- Removed irrelevant formatting such as headers, footers, and HTML tags.
- Normalized whitespace and special characters.

Chunking

- The document was split into **100–300 word chunks** using **sentence-aware splitting** to preserve context.

Embeddings & Storage

- Generated embeddings using **all-MiniLM-L6-v2** (from SentenceTransformers).
- Indexed embeddings into a **FAISS vector database** for efficient semantic search.

Model Fine-Tuning / Prompt Tuning

Model Selection

- Selected **Mistral-7B-Instruct** as the base model for generation due to its efficiency and instruction-following capabilities.

Prompt Template

A carefully designed **prompt template** ensures the model generates factual and grounded answers:

```
You are a helpful assistant. Use the provided context to answer the question
factually.
If the context does not contain the answer, say "The information is not
available in the document."

Context:
{retrieved_chunks}

Question: {user_query}

Answer:
```

RAG Pipeline Implementation

Retriever

- Used **semantic similarity search** over FAISS to retrieve the top relevant chunks.

Generator

- Injected retrieved chunks + user query into the **prompt template**.
- Generated responses with **Mistral-7B-Instruct**.

Response Handling

- Responses are **streamed in real-time** (sentence-by-sentence).
- Displayed **source chunks** to improve transparency.

Streamlit Chatbot Deployment

Features Implemented

- **User Input Field** for natural queries.
- **Real-Time Streaming Responses** for interactive experience.
- **Source Text Display** (retrieved chunks shown under each response).
- **Sidebar Information**:
    - Current Model in Use
    - Number of Indexed Chunks
- **Reset Functionality** to clear the chat.

Results & Observations

- The chatbot provides **factually grounded responses** based on document context.
- Sentence-aware chunking improved **retrieval accuracy**.
- Streaming responses improved **user experience**.
- Model was able to **cite references** consistently.

Future Improvements

- Use **hybrid retrieval** (semantic + keyword search).
- Experiment with **bge-large-en** embeddings for better recall.
- Add **feedback mechanism** to fine-tune prompt strategies.
- Deploy on **cloud infrastructure** for scalability.

Conclusion

This project successfully demonstrates the end-to-end development of a **RAG Chatbot** with document ingestion, retrieval, generation, and real-time deployment. It highlights the potential of combining **vector databases** with **open-source LLMs** to build practical, domain-specific assistants.